

1 UNIVERSITY OF EDINBURGH
2 SCHOOL OF INFORMATICS

3 INTERNSHIP REPORT

4 M1 INFORMATIQUE CONCEPTS ET APPLICATIONS

5
6 **Varieties of Semiring Solvers via**
7 **Multicategorical and Algebraic Constructions**

8

9 *Intern:*
Amaury MAZOYER
ENS de Lyon,
France

Supervisor:
Ohad KAMMAR
University of Edinburgh,
United Kingdom

10 May 4 – July 31 2026



THE UNIVERSITY of EDINBURGH
informatics

1 Introduction

In theorem provers and dependently-typed programming languages, users often have to prove to a type checker that two terms are equivalent. These proofs often involves repetitive application of the axioms defining the underlying theory. For instance, a proof of the classic identity

$$\forall a, b \in \mathbb{R}, (a + b)^2 = a^2 + 2ab + b^2$$

invokes 10 semiring axioms: associativity of the addition three times, commutativity of the multiplication once, left distributivity of times over plus three times, left neutrality of the multiplication twice and evaluation once.

In order to free users from needing to construct such proofs, most dependently typed languages include simplifiers for common algebraic structures. Among these structures are semirings and rings. A semiring consists of a set with two binary operations, typically called addition and multiplication, such that the ring is a commutative monoid with respect to addition, a monoid with respect to multiplication and multiplication distributes over addition. The set of natural numbers equipped with the usual operations form a semiring. A ring is a semiring that is an abelian group with respect to addition, i.e., each element must have an additive inverse. The set of integers equipped with the usual operations form a ring. Solvers for semirings and rings already exist and are implemented in most proof assistants. The state-of-the-art in terms of ring solvers is Rocq’s current implementation of its `ring` tactic [Grégoire & Mahboubi, 2005]. This is also the implementation chosen by Agda [Kidney, 2019]. The technique used is called reflection, and while Grégoire and Mahboubi unify the implementation for semirings and rings, it is limited to those theories. For instance, one would need to implement a new solver from the ground up to deal with involutive rings.

The goal of this internship is to find a way to combine existing solvers for component theories to obtain a solver for distributive combinations of theories. Doing so would let us implement a semiring solver from a monoid solver and a commutative monoid solver. Other than letting us implement solvers more easily, this also opens the door to a wide variety of other semiring solvers, considering combinations of varieties of semigroups, monoids and groups.

This internship is part of a wider project called **free extensions**, abbreviated FREX. Introduced in Yallop et al. [2018], and built on universal algebra, it is first thought as a way to represent partially-static structures, where some parts of a data structure are known at compile-time (static) and other parts are known only at run-time (dynamic). FREX relies on *normal forms*, i.e., canonical representatives using the algebraic laws while also evaluating concrete elements. The first application of FREX was multi-stage programming (MSP), a form of metaprogramming where parts of source code are annotated and become available as symbolic expressions. FREX can also be specialised to fully dynamic structures, considering free algebras (abbreviated *fral*) instead of free extensions. With its use of normal forms, FREX represents terms modulo semantics. This leads to so-called ‘representation theorems’ which can be used for algebraic simplification thanks to universal algebra. The use of normal forms in the theory may also be useful for search and synthesis, by enumerating candidates over a smaller search-space.

Later, Allais et al. [2025] formalised these representation theorems constructively and with respect to setoid, alongside other universal algebra concepts. This let them implement an extensible dependently typed library for algebraic simplifiers based on *fral* and *frex* representation theorems.

FREX can be extended modularly: Allais et al. [2023] describes a modular construction of an involutive algebra *frex* out of the *frex* of the underlying algebra. This lets them expand for

free on already constructed free extensions, and combined with the work of Allais et al. [2025], this gives solvers for involutive structures for free. For instance, implementing a simple monoid solver using frex yields an involutive monoid solver. This motivates the search for a modular construction for solvers for distributive combination of theories using FREX.

Reformulated using FREX concepts, the goal of this internship is to find a way to combine free algebras and free extensions for component theories to obtain free algebras/extensions for distributive combination of theories.

1.1 Outline and contributions

In §2, we present the relevant mathematical concepts behind FREX, how they are used in practice to simplify equations and also some concepts of category theory that are used to state the main result.

In §3, we present the theoretical construction of free algebras for the distributive combination of theories. Ohad Kammar had stated an incomplete result (§3.2), but it turns out it contradicted known results (§3.3). I combed the (incomplete) proof for errors, strengthened the hypotheses to state a new corrected result and proved it (§3.4). We started extending the result from free algebras to free extensions, but the construction is incomplete (§3.5).

In §4, we briefly present the implementation of the distributive combination of free algebras in Idris 2, what remains to be implemented, as well as a full performance evaluation (§4.4). To do so, I also had to implement some additional solvers for the component theories (§4.3).

In §5, we compare different aspects of FREX with the state-of-the-art in terms of semiring solvers: Rocq’s reflection-based ring tactic [Kidney, 2019]. §5.1 and §5.2 give insights into how the ring tactic works, while §5.3 explains the soundness and completeness guarantees of both libraries. §5.4 compares the capabilities and applications of FREX and the ring tactic.

The appendices contain some additional definitions and proofs (§A), the Idris 2 source code for the whole project (§B), the full proof of Theorem 4 (§C) and a link to my research notes (§D).

2 Preliminaries

FREX is based on some concepts of *universal algebra*, which gives generic descriptions of algebraic structures and relations between them [Burris & Sankappanavar, 1981, Chp. II]. We’ll summarise the relevant concepts, which have been formalised in Idris 2 [Allais et al., 2025]. We’ll use monoids to illustrate these concepts along the way. We’ll also explain how simplification works, and we’ll define the distributive combination of theories formally.

2.1 Syntax

A *finitary signature* $\Sigma = (\text{op } \Sigma, \text{arity})$ consists of a set $\text{op } \Sigma$ of *operators*, and an assignment $\text{arity} : \text{op } \Sigma \rightarrow \mathbb{N}$ associating to each operator in $\text{op } \Sigma$ its *arity*. The usual signature Σ_{Mon} of monoids consists of two operators $(\cdot)$ and 1 , with respective arities 2 and 0. We’ll write $\Sigma_{\text{Mon}} := \{(\cdot) : 2, 1 : 0\}$.

An *algebra* $A = (\underline{A}, A \llbracket - \rrbracket)$ over a signature Σ consists of a set $\underline{A}$ called the *carrier* of A , and a function $A \llbracket f \rrbracket : \underline{A}^n \rightarrow \underline{A}$ called the *interpretation* of f in A for each operator $(f : n) \in \Sigma$. Interpretations give the semantics to the algebraic language determined by the signature. There may be different algebras for a given signature and carrier set. One can equip the set $\mathbb{N}$ of natural numbers with the monoid structure $(\mathbb{N}, (+), 0)$, $(\mathbb{N}, (\cdot), 0)$ or even $(\mathbb{N}, \text{max}, 0)$.

Given a set X of variables and a signature Σ , the set of Σ -terms over X , denoted $\text{Term}_\Sigma(X)$, is defined by induction:

$$\frac{x \in X}{\text{var } x \in \text{Term}_\Sigma(X)} \quad \frac{t_i \in \text{Term}_\Sigma(X) \quad (f : n) \in \Sigma}{f(t_1, \dots, t_n) \in \text{Term}_\Sigma(X)}$$

We'll usually write $x \in \text{Term}_\Sigma(X)$ instead of $\text{var } x \in \text{Term}_\Sigma(X)$. We define *equations in context* $X \vdash l = r$ as triples (X, l, r) where X is a set of variables and l, r are terms in context X called the *left-hand-side* (LHS) and *right-hand-side* (RHS). The associativity equation in Σ_{Mon} is then defined as $x, y, z \vdash x \cdot (y \cdot z) = (x \cdot y) \cdot z$.

An *environment* ρ for a context X , or X -*environment*, in an algebra A is a function $\rho : X \rightarrow \underline{A}$ that assigns to each variable in X an element of the carrier of A . Given a term $t \in \text{Term}_\Sigma(X)$, denoted $X \vdash t$, the algebra A gives an *interpretation* function $A \llbracket t \rrbracket : \underline{A}^X \rightarrow \underline{A}$. For an X -environment ρ , $A \llbracket t \rrbracket \rho$ is defined by induction:

$$A \llbracket \text{var } x \rrbracket \rho := \rho x \quad A \llbracket \text{op}(t_1, \dots, t_n) \rrbracket \rho := A \llbracket \text{op} \rrbracket (A \llbracket t_1 \rrbracket \rho, \dots, A \llbracket t_n \rrbracket \rho)$$

For example, in the Σ_{Mon} -algebra $A = (\mathbb{N}, (+), 0)$, one can interpret the term $x \cdot (y \cdot 1)$ in the environment $\{x \mapsto 1, y \mapsto 2\}$, yielding $A \llbracket x \cdot (y \cdot 1) \rrbracket = 1 + (2 + 0) = 3$

An algebra A paired with an X -environment $\rho : X \rightarrow \underline{A}$ satisfies an equation $X \vdash l = r$ between terms in $\text{Term}_\Sigma(X)$ when $A \llbracket l \rrbracket \rho = A \llbracket r \rrbracket \rho$. We write this $(A, \rho) \models (X \vdash l = r)$. The equation $X \vdash l = r$ is *valid* in A when every environment satisfies it,

$$\forall \rho : X \rightarrow \underline{A}, \quad (A, \rho) \models (X \vdash l = r)$$

We write this $A \models (X \vdash l = r)$. For example, the Σ_{Mon} algebras presented so far validate the associativity axiom, but interpreting the binary operation as subtraction over the integers $(\mathbb{Z}, (-), 0)$ does not: taking the environment $\{x \mapsto 0, y \mapsto 0, z \mapsto 1\}$, we have $0 - (0 - 1) = 1 \neq -1 = (0 - 0) - 1$.

A *presentation* $\mathcal{T} = \langle \Sigma_{\mathcal{T}}, \text{Ax}_{\mathcal{T}} \rangle$ consists of a signature $\Sigma_{\mathcal{T}}$ and a set $\text{Ax}_{\mathcal{T}}$ of Σ -equations in context. A $\mathcal{T}$ -*model* A is a $\Sigma_{\mathcal{T}}$ algebra validating all of the axioms in $\text{Ax}_{\mathcal{T}}$. The **Monoid** presentation consists of the associativity axiom paired with the neutrality axioms $x \vdash x \cdot 1 = x$ and $x \vdash 1 \cdot x = x$ over the signature Σ_{Mon} . The Σ_{Mon} -algebras $(\mathbb{N}, (+), 0)$, $(\mathbb{N}, (\cdot), 0)$ and $(\mathbb{N}, \max, 0)$ are all **Monoid**-models.

Let $A = (\underline{A}, A \llbracket - \rrbracket)$ and $B = (\underline{B}, B \llbracket - \rrbracket)$ be algebras over the same signature Σ . A *homomorphism* of Σ -algebras from A to B is a function $h : \underline{A} \rightarrow \underline{B}$ such that for every operator $f : n \in \Sigma$ and every $a_1, \dots, a_n \in \underline{A}$ we have:

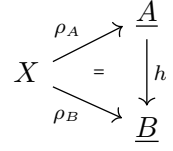
$$B \llbracket f \rrbracket (h(a_1), \dots, h(a_n)) = h(A \llbracket f \rrbracket (a_1, \dots, a_n))$$

In other words, a homomorphism is a semantics-preserving function between the carriers of the algebras. For example, complex conjugation is a homomorphism between the Σ_{Mon} -algebras over the complex numbers, since $\overline{z_1 + z_2} = \overline{z_1} + \overline{z_2}$, $\overline{0} = 0$ and $\overline{z_1 \cdot z_2} = \overline{z_1} \cdot \overline{z_2}$, $\overline{1} = 1$. A *homomorphism* of $\mathcal{T}$ -models is a homomorphism of $\Sigma_{\mathcal{T}}$ -algebras between the underlying algebras of the models.

2.2 Free models/algebras

Let $\mathcal{T}$ be a presentation and X a set of variables. A $\mathcal{T}$ -model over X amounts to a pair $\langle A, e \rangle$ where A is a $\mathcal{T}$ -model and $e : X \rightarrow \underline{A}$ is an X -environment in this algebra. This concept is used to formalise the free algebra (frl) simplification in §2.4.

119 A morphism $h : \langle A, \rho_A \rangle \rightarrow \langle B, \rho_B \rangle$ between $\mathcal{T}$ -models over X is a $\mathcal{T}$ -model
 120 homomorphism that makes the diagram on the right commute. A *free $\mathcal{T}$ -model*
 121 *over X* is a $\mathcal{T}$ -model $\langle FX, \text{var} \rangle$ such that for every $\mathcal{T}$ -model A over X , there
 122 exists a unique morphism of $\mathcal{T}$ -models from $\langle FX, \text{var} \rangle$ to A . We will often use
 123 the term “free algebra”, abbreviated “fral”, when $\mathcal{T}$ is implicit. The environment
 124 var is called the *unit* of the free algebra. The existence-and-uniqueness property
 125 is called the *universal property* of the free algebra. Thanks to the universal property, all free
 126 $\mathcal{T}$ -models over X are uniquely isomorphic and therefore we often talk about *the* free algebra
 127 over X . Informally, the free algebra for a theory is the most parsimonious model validating
 128 the theory, in the sense that the only equations validated by the free algebra are the ones from
 129 the presentation and the ones that we can deduce from them. Free algebras represent normal
 130 forms of terms modulo the equations of the theory. The free commutative monoid over the set
 131 of variables $\{x_1, x_2, x_3\}$ can be represented by $\mathbb{N}^3$. Addition is coordinate wise, the 0 element is
 132 $(0, 0, 0)$ and var is the function that sends x_1 to $(1, 0, 0)$, x_2 to $(0, 1, 0)$ and x_3 to $(0, 0, 1)$. The
 133 unique morphism out of this algebra into any commutative monoid A is the function sending
 134 (a_1, a_2, a_3) to $a_1(\rho_A(x_1)) + a_2(\rho_A(x_2)) + a_3(\rho_A(x_3))$. For monoids, the free monoid over a
 135 set of variables X can be represented as finite lists of elements of X , with concatenation as
 136 multiplication and where variables are injected as singleton lists.



137 Every presentation $\mathcal{T}$ induces an equivalence relation between terms using the deduction
 138 rules of equational logic. Explicitly, we define a *provability* relation $X \vdash_{\mathcal{T}} l = r$ inductively as
 in Figure 1. Derivations are built similarly to natural deduction.

$$\begin{array}{c}
 \frac{}{X \vdash_{\mathcal{T}} t = t} \quad (\text{reflexivity}) \qquad \frac{X \vdash_{\mathcal{T}} l = r}{X \vdash_{\mathcal{T}} r = l} \quad (\text{symmetry}) \\
 \frac{X \vdash_{\mathcal{T}} l = m \quad X \vdash_{\mathcal{T}} m = r}{X \vdash_{\mathcal{T}} l = r} \quad (\text{transitivity}) \qquad \frac{(Y \vdash_{\mathcal{T}} l = r) \in \text{Ax}_{\mathcal{T}} \quad \theta : Y \rightarrow \text{Term}_{\Sigma_{\mathcal{T}}} X}{X \vdash_{\mathcal{T}} l[\theta] = r[\theta]} \quad (\text{axiom}) \\
 \frac{\text{for } i = 1, \dots, n : X \vdash_{\mathcal{T}} l_i = r_i}{X \vdash_{\mathcal{T}} \text{op}(l_1, \dots, l_n) = \text{op}(r_1, \dots, r_n)} \quad (\text{congruence})
 \end{array}$$

Figure 1: Provability in equational logic

139

There is an isomorphism of $\mathcal{T}$ -models over X

$$\langle FX, \text{var}_{FX} \rangle \cong \langle (\text{Term}_{\Sigma_{\mathcal{T}}} X) / (X \vdash_{\mathcal{T}}) =: Q, \text{var}' \rangle$$

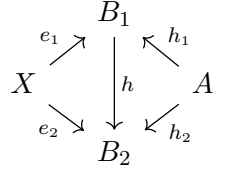
140 with $\text{var}' : X \rightarrow Q$ given by $x \mapsto [\text{var } x]$. Therefore free algebras represent terms modulo
 141 semantics.

142 2.3 Free extensions

143 Free algebras allow us to deal with dynamic equations that involve *only* free variables, but some-
 144 times one may need to simplify equations that involve partially-static terms, i.e, terms with vari-
 145 ables and constants, such as $2 + x + y$ for instance. The analog of free algebras for partially-static
 146 structures are free extensions.

147 An *extension of A by X* is a triple $\langle B, h, e \rangle$ where B is a $\mathcal{T}$ -algebra, $h : A \rightarrow B$ is a
 148 $\mathcal{T}$ -homomorphism and $e : X \rightarrow B$ is a function.

Let $b_1 = \langle B_1, h_1, e_1 \rangle, b_2 = \langle B_2, h_2, e_2 \rangle$ be two extensions of A by X . A morphism of extensions $h : b_1 \rightarrow b_2$ is a $\mathcal{T}$ -homomorphism $h : B_1 \rightarrow B_2$ that makes the diagram on the right commute. An extension $\langle A[X], \text{sta}, \text{dyn} \rangle$ is free when, for every extension $\langle B, h, e \rangle$, there is a unique extension morphism $[B, h, e] : \langle A[X], \text{sta}, \text{dyn} \rangle \rightarrow \langle B, h, e \rangle$. Free extension is abbreviated “frex”, and we denote free extensions of A by X by $A[X]$. Just like for free algebras, all free extensions of A by X are isomorphic to each other. Let A be a commutative monoid. Its frex by the finite set of variables $\{x_1, x_2, x_3\}$ can be represented by $A[X] := \underline{A} \times \mathbb{N}^3$. The remaining structure is defined by:



$$\begin{aligned} \langle a, \langle k_{11}, k_{12}, k_{13} \rangle \rangle + \langle b, \langle k_{21}, k_{22}, k_{23} \rangle \rangle &:= \langle a + b, \langle k_{11} + k_{21}, k_{12} + k_{22}, k_{13} + k_{23} \rangle \rangle \\ 0 &:= \langle 0, \langle 0, 0, 0 \rangle \rangle \quad \text{sta } a := \langle a, \langle 0, 0, 0 \rangle \rangle \quad \text{dyn } x_1 := \langle 0, \langle 1, 0, 0 \rangle \rangle \quad \text{dyn } x_i := \dots \\ [B, h, e] \langle a, \langle k_1, k_2, k_3 \rangle \rangle &:= h a + k_1 \cdot e x_1 + k_2 \cdot e x_2 + k_3 \cdot e x_3 \end{aligned}$$

Similarly to §2.2, there is a canonical isomorphism between any free extension and the quotient frex $((\text{Term}_{\Sigma_{\mathcal{T}_A}} X) / (X \vdash_{\mathcal{T}_A}) =: Q, \text{sta}', \text{dyn}')$ with $\text{sta}' : A \rightarrow Q$ defined by $a \mapsto [\underline{a}]$ and $\text{dyn}' : X \rightarrow Q$ defined by $x \mapsto [\text{var } x]$. $\mathcal{T}_A$ is the presentation where the operators are the ones of $\mathcal{T}$ together with newly adjoined constants $\underline{a}$ for every $a \in \underline{A}$, and whose axioms are the combination of the axioms in $\mathcal{T}$ and the evaluation axioms $\vdash \text{op}(\underline{a}_1, \dots, \underline{a}_n) = A \llbracket \text{op} \rrbracket (a_1, \dots, a_n)$ for every $\text{op} : n \in \Sigma_{\mathcal{T}}$ and $(a_1, \dots, a_n) \in \underline{A}^n$ [Allais et al., 2023]. In this way, each free extension is also a free algebra of a theory specialised to a concrete algebra by adding axioms over the concrete elements.

2.4 How fral/frex solving works

We explain how fral solving works, and then how to adapt it to free extensions. This has been formalised in Idris 2 [Allais et al., 2025], see §4.

Suppose that we want to prove that the equation $x + (y + x) + y = x + x + y + y$ holds in a commutative monoid A . We evaluate the LHS and RHS of the equation in the free algebra, both give the representative $(2, 2)$. We then prove the equality in the free algebra, this is simple reflexivity here, and we use the universal property of the free algebra to conclude that the equation is valid in A .

In the general setting, let X be a set of variables, $\mathcal{T}$ be a theory and $\langle FX, \text{var}_{FX} \rangle$ be a free $\mathcal{T}$ -model over X . We have the following theorem

Theorem 1 (extensional normalisation)

Let $X \vdash l = r$ be an equation in context.

The following are equivalent:

1. The unit satisfies the equation: $(FX, \text{var}_{FX}) \models (X \vdash l = r)$
2. The free algebra validates the equation: $FX \models (X \vdash l = r)$
3. Every $\mathcal{T}$ -model A validates the equation: $A \models (X \vdash l = r)$

Proof

This is proved in §A. □

Proving equalities in arbitrary models with free algebras then boils down to proving equalities in the free algebra. Since free algebras are canonically isomorphic to the quotient algebra

178 of terms quotiented by the provability relation, if an equality is indeed provable, then the two
 179 associated terms in the free algebra should also be equal.

180 Since free algebras represent normal forms of terms modulo the equations of the theory, if
 181 the two terms of the equation are indeed equivalent in the theory, then the interpretation of
 182 those terms with the unit of the free algebra must return identical terms. The only difficulty lies
 183 in the definition of equality in the free algebra.

Proving equalities between partially static terms using free extensions is similar and reuses
 the same construction. We use the fact that a free extension is a free algebra for a specific theory
 as mentioned in §2.3. Instead of considering an equation on X , we consider an equation on
 $X \sqcup \underline{A}$. For an extension $\langle A, \text{sta}, \text{dyn} \rangle$, the environment we use is defined as follows:

$$\text{var } x := \begin{cases} \text{sta } x & \text{if } x \in \underline{A} \\ \text{dyn } x & \text{if } x \in X \end{cases}$$

184 Using the same reasoning as before and the universal property of the free extension, we can
 185 prove partially static equalities on arbitrary models.

186 2.5 The distributive combination of theories

Given two signatures Σ_1 and Σ_2 , we define their coproduct $\Sigma_1 + \Sigma_2$ as the signature with oper-
 ators the disjoint union of the operators, but retaining their arity:

$$\Sigma_1 + \Sigma_2 = \{\iota_i f : n \mid (f : n) \in \Sigma_i, i = 1, 2\}$$

187 Given a Σ_i -term-in-context $n \vdash_{\Sigma_i} t$, we denote by $n \vdash_{\Sigma_1 + \Sigma_2} \iota_i[t]$ the $\Sigma_1 + \Sigma_2$ -term-in-context
 188 obtained by applying the injection ι_i to all the operators of t . Similarly, given a Σ_i -equation in
 189 context $\text{eq} := (n \vdash_{\Sigma_i} t = t')$, we denote by $\iota_i[\text{eq}]$ the $\Sigma_1 + \Sigma_2$ -equation in context
 190 $n \vdash_{\Sigma_1 + \Sigma_2} \iota_i[t] = \iota_i[t']$.

191 Let $\mathcal{A}$ and $\mathcal{M}$ be two theories. We define the *distributive combination of $\mathcal{M}$ over $\mathcal{A}$* [Hyland
 192 & Power, 2006] to be the theory $\mathcal{M} \triangleright \mathcal{A}$ given by:

$$\begin{aligned} \Sigma_{\mathcal{M} \triangleright \mathcal{A}} &:= \Sigma_{\mathcal{M}} + \Sigma_{\mathcal{A}} \\ \text{Ax}_{\mathcal{M} \triangleright \mathcal{A}} &:= \iota_1[\text{Ax}_{\mathcal{M}}] \cup \iota_2[\text{Ax}_{\mathcal{A}}] \\ &\cup \left\{ \begin{array}{l} \langle x_i \rangle_{\substack{i=1 \\ i \neq i_0}}^n, \mathbf{y} \mid f(x_1, \dots, x_{i_0-1}, s\mathbf{y}, x_{i_0+1}, \dots, x_n) \\ \quad = s \langle f(x_1, \dots, x_{i_0-1}, \mathbf{y}_j, x_{i_0+1}, \dots, x_n) \rangle_{j=1}^{|\mathbf{y}|} \end{array} \mid \begin{array}{l} (f : n) \in \Sigma_{\mathcal{M}} \\ (s : |\mathbf{y}|) \in \Sigma_{\mathcal{A}} \end{array} \right\} \end{aligned}$$

193 Concretely, the signature is the coproduct signature and the axioms are the union of the axioms
 194 to which we added distributivity axioms between the operators of $\mathcal{M}$ and $\mathcal{A}$, stating that every
 195 operator of $\mathcal{M}$ distributes over every operator of $\mathcal{A}$. We will call $\mathcal{M}$ the multiplicative theory,
 196 and $\mathcal{A}$ the additive theory. For example, the theory of semirings is the distributive combination
 197 of commutative monoids over monoids. The distributivity axioms when unfolded state that:

$$\begin{aligned} x, y, z \vdash x \cdot (y + z) &= (x \cdot y) + (x \cdot z) & x, y, z \vdash (x + y) \cdot z &= (x \cdot z) + (y \cdot z) \\ x \vdash x \cdot 0 &= 0 & x \vdash 0 \cdot x &= 0 \end{aligned}$$

198 The multiplicative operation acts linearly in each argument with respect to the additive theory,
 199 and constants of the additive theory taken as operators give annihilation laws.

2.6 Monads and distributive laws

Monads are closely related to algebraic theories, and the construction of the distributive combinations in §3 uses the notions of monads and distributive laws from a category-theoretical point of view. We will assume basic knowledge about category theory, mainly about functors, natural transformations and adjunctions. One can refer to MacLane [1971].

A *monad* on a category $\mathcal{C}$ is a triple (T, η, μ) where $T : \mathcal{C} \rightarrow \mathcal{C}$ is a functor, $\eta : \text{Id}_{\mathcal{C}} \rightarrow T$ is a natural transformation called the unit, $\mu : T^2 \rightarrow T$ is a natural transformation called the multiplication, and such that the following diagrams commute:

$$\begin{array}{ccc} T & \xrightarrow{T\eta} & T^2 \\ \eta T \downarrow & \searrow & \downarrow \mu \\ T^2 & \xrightarrow{\mu} & T \end{array} \quad \begin{array}{ccc} T^3 & \xrightarrow{T\mu} & T^2 \\ \mu T \downarrow & & \downarrow \mu \\ T^2 & \xrightarrow{\mu} & T \end{array}$$

Every adjunction $\langle F, G, \eta, \varepsilon \rangle$ gives rise to a monad [MacLane, 1971], by defining $T := GF$, $\eta := \eta$ and $\mu := G\varepsilon F$.

Theorem 2

For every presentation $\mathcal{T}$, there is a left adjoint $F^{\mathcal{T}}$ to the obvious forgetful functor $U^{\mathcal{T}} : \mathcal{T}\text{-Alg} \rightarrow \mathbf{Set}$ where $\mathcal{T}\text{-Alg}$ is the category with $\mathcal{T}$ -models as objects and $\mathcal{T}$ -model homomorphisms as morphisms. The functor is defined as follows:

- For a set X , $F^{\mathcal{T}}X := \underline{F_{\mathcal{T}}X}$ is the carrier of the free $\mathcal{T}$ -algebra over X .
- The action on a function $h : X \rightarrow Y$ is defined inductively by:

$$\begin{aligned} F^{\mathcal{T}}(h)(x) &= h(x) \quad \text{for } x \in X \\ F^{\mathcal{T}}(h)(\text{op}(t_1, \dots, t_n)) &= \text{op}(F^{\mathcal{T}}(h)(t_1), \dots, F^{\mathcal{T}}(h)(t_n)) \quad \text{for } \text{op} : n \in \Sigma_{\mathcal{T}} \end{aligned}$$

This is a standard theorem and can be found in Zwart and Marsden [2018].

For a presentation $\mathcal{T}$, the *free model monad* induced by $\mathcal{T}$ is then the monad induced by the free/forgetful adjunction $U^{\mathcal{T}} \circ F^{\mathcal{T}}$. We say that a monad T corresponds to a presentation $\mathcal{T}$ if T is a free model monad induced by $\mathcal{T}$. It is well known that every finitary monad on the category of sets arises as a free model monad for some algebraic theory. For instance, the monad T corresponding to the algebraic theory $\mathcal{T}$ is the monad whose terms TX are terms over the variables in X modulo the equations, whose unit $\eta_X : X \rightarrow TX$ sends each variable to the corresponding atomic term and whose multiplication $\mu_X : TTX \rightarrow TX$ is substitution/flattening, i.e., the act of plugging terms into terms. For the theory of (regular) monoids, the corresponding monad is the List monad whose elements LX are finite lists over X , $\eta_X^L(x) = [x]$ and μ_X^L is list concatenation.

Given two monads (T, η^T, μ^T) and (S, η^S, μ^S) on a category $\mathcal{C}$, a *distributive law* [Beck, 1969] of T over S is a natural transformation $\lambda : TS \rightarrow ST$ such that the following diagrams commute:

$$\begin{array}{ccc} & T & \\ T\eta^S \swarrow & & \searrow \eta^ST \\ TS & \xrightarrow{\lambda} & ST \end{array} \quad \begin{array}{ccc} & S & \\ \eta^TS \swarrow & & \searrow S\eta^T \\ TS & \xrightarrow{\lambda} & ST \end{array}$$

224

$$\begin{array}{ccc}
T^2S & \xrightarrow{T\lambda} & TST \xrightarrow{\lambda T} ST^2 \\
\mu^T S \downarrow & & \downarrow S\mu^T \\
TS & \xrightarrow{\lambda} & ST
\end{array}
\qquad
\begin{array}{ccc}
TS^2 & \xrightarrow{\lambda S} & STS \xrightarrow{S\lambda} S^2T \\
T\mu^S \downarrow & & \downarrow \mu^{ST} \\
TS & \xrightarrow{\lambda} & ST
\end{array}$$

225 These equations ensure that the distributive law is compatible with both the mul-
 226 tiplications of S and T . This law induces a composite monad ST on $\mathcal{C}$ with unit $\eta^{ST} = \eta^S \eta^T$
 227 and multiplication $STST \xrightarrow{S\lambda T} SSTT \xrightarrow{\mu^S \mu^T} ST$.

The most famous example of a distributive law involves the List monad distributing over the Abelian group monad (Definition 7): $\lambda : LA \Rightarrow AL$ [Beck, 1969]. It captures the classic distributivity of multiplication over addition. Simply put,

$$\lambda(a \cdot (b + c)) = a \cdot b + a \cdot c$$

We can write lists as formal products and multisets as formal sums, letting us write the following definition for λ :

$$\lambda \left(\prod_{i=0}^n \sum_{j_i=0}^{m_i} x_{\langle i, j_i \rangle} \right) = \sum_{j_0=0}^{m_0} \cdots \sum_{j_n=0}^{m_n} \prod_{i=0}^n x_{\langle i, j_i \rangle}$$

228 The resulting composite monad is the ring monad, i.e the monad corresponding to the algebraic
 229 theory of rings. This is the essence of what we will use to construct the free algebra for the
 230 distributive combination of theories.

231 3 Categorical approach to the construction of distributive com- 232 binations

233 While using FREX does not require knowledge about category theory, FREX uses many category-
 234 theoretic concepts internally. Therefore, the construction of distributive combinations is pri-
 235 marily category-theoretic. However, we do not go into the details here, as this would require a
 236 course on 2-categories and multicategories.

237 3.1 Challenge about constructing distributive combinations

238 Constructing generic free algebras and free extensions for arbitrary theories is not hard. In fact,
 239 §2.2 gives an explicit construction for the free algebra of any theory using quotients. However in
 240 order to simplify equations computationally, §2.4 shows that we need a normal form with decid-
 241 able equality, not merely an abstract quotient construction. The equivalence relation between
 242 terms in the quotient is not necessarily effective/decidable. Without this, we won't be able to
 243 prove equalities in the free algebra, and therefore to prove any equation with free algebras.

244 Free extensions also have a generic construction: Allais et al. [2025] and Yallop et al. [2018]
 245 state that the free extension of an algebra A by X is the category-theoretic coproduct of A
 246 with the free $\mathcal{A}$ -algebra over X . However, there is no generic formula for the coproduct of two
 247 algebras for an arbitrary theory.

248 The difficulty of the construction is for it to be effective so that equivalence between two
 249 terms can be determined computationally. This forbids the use of quotients, category theory
 250 pushouts etc...

251 3.2 Initial result on the free algebra for the distributive combination

252 A first hypothesis was given by Ohad Kammar (my internship supervisor) about the free algebra
 253 for the distributive combination of a theory over a commutative theory. A theory $\mathcal{A}$ is commu-
 254 tative iff every operation of the theory commutes with every other operations, or more formally,
 255 iff for every $(f : n) \in \Sigma_{\mathcal{A}}$ and every $(g : m) \in \Sigma_{\mathcal{A}}$,

$$f \left(\begin{array}{c} g(x_{11}, \dots, x_{1m}) \\ \vdots \\ g(x_{n1}, \dots, x_{nm}) \end{array} \right) = g \left(f \left(\begin{array}{c} x_{11} \\ \vdots \\ x_{n1} \end{array} \right), \dots, f \left(\begin{array}{c} x_{1m} \\ \vdots \\ x_{nm} \end{array} \right) \right) \quad (1)$$

256 This definition is stronger and more structural than ordinary commutativity of a single binary
 257 operation. For monoids, this states that

$$(x \cdot y) \cdot (z \cdot w) = (x \cdot z) \cdot (y \cdot w) \quad 0 \cdot 0 = 0 \quad 0 = 0$$

258 The theory of commutative monoids is therefore a commutative theory, fortunately. Note that
 259 every operator of arity 0 and 1 commutes with itself.

260 The initial result was formulated in terms of multicategories, we reformulate it here in terms
 261 of monads for ease of understanding.

Hypothesis 3 (Free algebra for the distributive combination, incorrect)

Let $\mathcal{A}$ be a commutative theory, and $\mathcal{M}$ be any theory. Then there exists a distributive law of the free model monad induced by $\mathcal{M}$ over the free model monad induced by $\mathcal{A}$, and the free model monad induced by $\mathcal{M} \triangleright \mathcal{A}$ is the composite monad of the two free model monads induced by the distributive law.

262 In the semiring case, this says that if $F_{\mathcal{M}}X$ gives multiplicative normal forms and $F_{\mathcal{A}}X$ gives
 263 additive normal forms, then $F_{\mathcal{A}}F_{\mathcal{M}}X$ gives normal forms for the semiring over X .

264 Intuitively, the hypothesis says that given two free algebras for the additive and multiplica-
 265 tive part, composing them gives the free algebra for the distributive combination of the theories.
 266 This construction is effective in the sense of §3.1.

267 Even though the construction itself was well-defined, the proof wasn't complete, and for a
 268 good reason: the statement is incorrect. The culprit is the absence of distributive laws under
 269 some hypothesis.

270 3.3 Why the initial result fails

271 Zwart and Marsden [2018] give multiple results about the absence of distributive laws under
 272 some hypothesis. We can regroup these results under 3 categories:

- 273 1. theorems 3.5, 3.12 and 4.24 all forbid the existence of an idempotent operator in the mul-
 274 tiplicative theory, on top of other conditions.
- 275 2. theorem 4.7 forbids the existence of two distinct constants in the additive theory.
- 276 3. theorem 4.17 imposes the abides equation $(a * b) * (c * d) = (a * c) * (b * d)$ to hold for
 277 every operator $*$ in the additive theory.

278 The second and third point are both taken care of by the hypothesis of commutativity of the
 279 additive theory in Hypothesis 3. For the second point, two constants commute with each other
 280 if they are equal. A commutative theory can therefore not have two or more distinct constants.
 281 For the third point, the abides equation is simply the commutativity hypothesis for $f = g = *$.

282 However the first point is not accounted for in Hypothesis 3. Informally, idempotence dupli-
 283 cates information in a way incompatible with the intended distributive-law construction. Under
 284 the current hypothesis, idempotent operators are allowed. Therefore, we need to strengthen our
 285 hypothesis.

286 A known failure is the distributive combination of semi-lattices with probabilistic nonde-
 287 terminism [Varacca & Winskel, 2006, due to Gordon Plotkin]. Let $\mathcal{M} = \mathbf{BA}$ be the theory of
 288 barycentric algebras (Definition 8), $\mathcal{A} = \mathbf{SL}$ be the theory of join-semilattices (Definition 9).
 289 Barycentric algebras have idempotent operators, therefore there cannot exist a distributive law
 290 $\lambda : \mathbf{BA} \mathbf{SL} \Rightarrow \mathbf{SL} \mathbf{BA}$. The free semilattice on a set X is $F_{\mathbf{SL}}X = \mathcal{P}_f^+(X)$ the set of non-empty
 291 finite subsets of X , with union as join, and the free barycentric algebra on X is $F_{\mathbf{BA}}X = \mathcal{D}_f(X)$
 292 the set of finitely-supported probability distributions on X . However, the free algebra on X for
 293 the distributive combination of the two is $\text{Conv}_f(\mathcal{D}_f(X))$, the set of non-empty finitely gen-
 294 erated convex subset of $\mathcal{D}_f(X)$, and not $\mathcal{P}_f^+(\mathcal{D}_f(X))$, the set of non-empty finite subsets of
 295 $\mathcal{D}_f(X)$.

296 3.4 Corrected result

297 We strengthen the hypothesis on $\mathcal{M}$ by requiring it to be an affine theory. A theory $\mathcal{M}$ is *affine*
 298 if every axiom is an equation between terms where each variable occurs at most once. This
 299 allows weakening/deletion of variables, but not contraction. For example, $x, y \vdash x + y = y + x$
 300 or $x \vdash x \cdot 0 = 0$ are affine equations, but $x \vdash x \vee x = x$ is not. Semigroups, monoids, groups and
 301 their commutative variants are all affine theories, whereas semilattices are not. This effectively
 302 covers the first point of the previous section.

303 We now state the corrected theorem, in terms of monads once again instead of multicate-
 304 gories.

Theorem 4 (Free algebra for the distributive combination)

Let $\mathcal{A}$ be a commutative theory, and $\mathcal{M}$ be an affine theory. Then there exists a distributive law of the free model monad induced by $\mathcal{M}$ over the free model monad induced by $\mathcal{A}$, and the free model monad induced by $\mathcal{M} \triangleright \mathcal{A}$ is the composite monad of the two free model monads induced by the distributive law.

305 The actual construction uses operads, translations from operads to theories, 2-categories and
 306 multicategories. Therefore, we won't present it nor the proof as is, but we'll explain the gist of
 307 it.

308 First, start by noticing that a model for the distributive combination is made of a $\mathcal{M}$ -model
 309 and a $\mathcal{A}$ -model on the same carrier set (this ensures that the respective axioms hold) where
 310 moreover the operations of $\mathcal{M}$ distribute over the operations of $\mathcal{A}$ (this is a pullback in the
 311 category of categories). Let X be a set of variables. The construction start by taking the free
 312 $\mathcal{M}$ -algebra over X , $A_0 := F_{\mathcal{M}}X$. For example, for monoids, $A_0 = F_{\mathbf{Mon}}X = \text{List } X$. We
 313 then take the free $\mathcal{A}$ -algebra over $F_{\mathcal{M}}X$, $A_1 := F_{\mathcal{A}}(F_{\mathcal{M}}X)$. For example, for commutative
 314 monoids, $A_1 = F_{\mathbf{CMon}}F_{\mathbf{Mon}}X = \text{Bag}(\text{List } X)$. This gives us our candidate $\mathcal{A}$ -model. This
 315 step corresponds to the monadic composition, minus the multiplications. But we now need to

define the semantics of the operators of $\mathcal{M}$ on the carrier $\underline{A}_1$. To do this, we “lift” the $\mathcal{M}$ -operators so that the lifted operators on the new carrier distribute over the $\mathcal{A}$ -operators. This is what gives the distributive law. For the distributive combination of monoids over commutative monoids, this yields the standard multiplication which distributes over addition. Through a more involved argument, and with the help of the affine hypothesis, we prove that the $\mathcal{M}$ -axioms hold. The commutativity hypothesis of $\mathcal{A}$ is used to prove the existence of an adjunction between multifunctors, which is used to lift the $\mathcal{M}$ -operators.

This construction is effective: it is solely built from applying the free algebra constructions and some 2-functors in well-chosen categories. The lifted operators can be defined concretely.

The description of the construction may not sound formal, but this is because we had to omit a lot of details for it to be understandable without the full multicategory and 2-category background.

3.5 An attempt at constructing the free extension for the distributive combination

The natural next step is to extend this result to free extensions. However the task is not so easy.

Since we have seen in §2.3 that a free extension is also a free algebra for a specialised theory, the first thought is to reuse our construction on this theory. However, the hypotheses do not hold anymore: the specialised theory contains a constant for every concrete element. As stated in §3.3, two constants commute with each other iff they are equal, and so the specialised theory for the free extension of any non-trivial algebra is not commutative. Therefore, we need to find a completely new construction.

Although the construction is nontrivial, it is not out of reach. We managed to define a construction using category-theoretic pushouts and the free extensions of the component theories. Although it is not effective, because pushouts are quotient-based, it shows that the free extensions of the component theories can indeed be used to construct the free extension of the combined theory. This led us to search for ways to adapt the construction for free algebras to free extensions. The construction of free extensions for distributive combinations is still ongoing work.

4 Implementation

FREX has multiple implementations, first in Haskell and MetaOCaml [Yallop et al., 2018], then in Agda [Corbyn, 2021] and finally in Idris 2 [Allais et al., 2025]. We are interested in the implementation in Idris 2, a purely functional programming language with first class types.

4.1 Setoids

FREX uses *setoids* [Bishop, 1967; Hofmann, 1997] instead of simple sets. This offers them some additional functionalities on top of term simplification and equation solving: printing terms and proofs, proof simplification, code generation and more [Allais et al., 2025].

A *setoid* is a set X equipped with an equivalence relation $\sim_X$. A *setoid homomorphism* $f : X \rightarrow Y$ is a function compatible with the equivalence relation: if $x \sim_X y$ then $f(x) \sim_Y f(y)$. Every set X can be seen as a setoid by equipping it with the equality relation ($=$). However setoids let us define some more complicated equivalence relations. For example, we can consider the setoids of terms for some theory $\mathcal{T}$ and set of variables X , equipped with the provability

relation from Fig 1. Two equivalent terms in this setoid represent different, but provably equal, terms. This is different from the quotient $(\text{Term}_{\Sigma_{\mathcal{T}}} X) / (X \vdash_{\mathcal{T}})$ where elements represent equivalence classes of provably equal terms.

FREX implements a slightly different version of the universal algebra presented in §2. Carriers for algebras are setoids instead of sets. Therefore, all homomorphisms must also be setoid homomorphisms.

The use of setoids may not seem justified yet, but it enables functionalities beyond algebraic solving in FREX, such as proof pretty-printing, code generation and more, which is explained in §5.4.3.

4.2 Implementing the distributive combination

The distributive combination for free algebras has been implemented modularly so that the user only needs to provide free algebras for the component theories to get a free algebra for the combined theory. These free algebras can be passed to dedicated functions of the Idris FREX library to act as solvers. These functions use a setoid variant of the extensional normalisation theorem (Theorem 1): FREX uses the implication $1. \Rightarrow 3.$ to prove the required equality. It tries to find a proof of 1. automatically, and asks the user for the proof if it fails. The proof should just be simple reflexivity, but Idris may not be able to solve it if the underlying equality is a user-defined setoid equivalence relation.

Unfortunately, not the entire distributive combination of free algebras has been implemented in this internship. Only the semantics have been implemented, it remains to prove in Idris 2 that the constructed algebra validates the axioms and that it is indeed free. This is due to the fact that formalising such a conceptual proof that stems directly from category theory is long and tedious. However, since we do have the full theoretical proof, we already have some guarantees about our result and we will be able to implement the remaining proofs, however long this may take.

However, these proofs are not needed for the actual solving logic to work; they merely guarantee that the solver behaves as intended. We were therefore able to use the current implementation to prove equations on various varieties of semirings. For instance, for every example of a distributive combination we implemented (see the Table 1), we proved that all of the axioms of the combination hold. This also lets us test the performance of our implementation in §4.4. But before being able to run some tests, we had to implement the missing solvers for the component theories, as only solvers for monoids and commutative monoids were implemented in Idris 2.

4.3 Implementing solvers

Implementing solvers for a given algebraic theory amounts to finding the free algebra or free extension for that theory and formalising it in Idris.

We implemented fral solvers for semigroups, commutative semigroups and abelian groups. Solvers for monoids, commutative monoids and involutive monoids were already implemented. For similar reasons as in §4.2, we only implemented the semantics of the free algebras and left the proofs for future work. This is enough to use these solvers to solve equations.

The free algebras for these theories are well-known, therefore implementing fral solvers is just a matter of formalising these objects in Idris. Free semigroups are non-empty lists, free commutative semigroups are finite multisets with non-empty support and free abelian groups are integer-valued finitely supported maps.

4.4 Performance evaluation

Following the work of Allais et al., 2025, we tested our implementation to find the instantaneity (0.1s), interactivity (1s) and attention (10s) thresholds described by Nielsen, 1994. This is important as developing in Idris2 is interactive and our tool is meant to discharge the user of painful arithmetic proofs. Each datapoint in Figure 2 is the median execution time on 10 random terms of that size. Term size is computed as the number of leaves in the syntax tree of the term.

FREX's type-checking time¹ remains below the instantaneity threshold for terms of size 60 to 120, depending on the number of free variables, as pictured in Figure 2. This is more than sufficient to deal with the typical equalities that arise in dependently typed programs.

FREX eventually crosses the interactivity threshold as the term sizes grow. However, by that point, we notice that some datapoints are missing. This happens because the test machine ran out of RAM on some of the test cases. This indicates that FREX's bottleneck is not its speed but Idris itself, and suggests that Frex is likely suitable for practical applications.

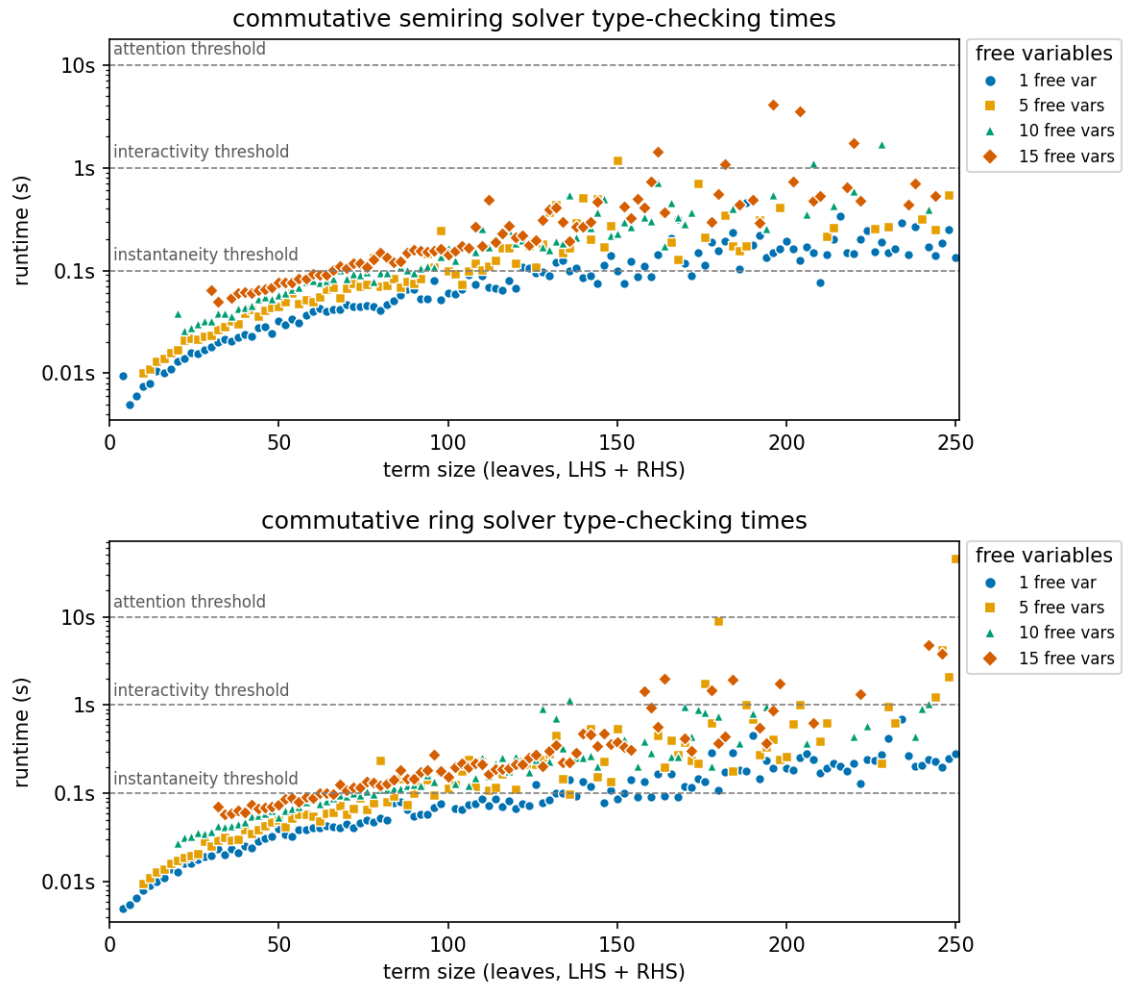


Figure 2: Type-checking times for the commutative semiring and ring solvers.

Comparing the execution times for semirings and rings with the ones in Allais et al. [2025] for monoids, the performance seems to have much improved. This can be explained by two different factors:

- Allais et al. [2025] mentioned a performance bottleneck in Idris2's evaluator, this was

¹We used an Intel Core Ultra 7 255HX machine with 16GB memory, running Idris 2 version 0.8.0-6a54860ee on Debian Linux 13.

during development four years ago and the development of Idris2 probably eliminated this bottleneck.

- The tests in Figure 2 have not been run on the same machine as the tests in Allais et al. [2025], and the CPU used for the semirings test is much more efficient than the one used for the monoids tests.

5 Comparison with existing solvers

In dependently-typed programming languages, the state-of-the-art for polynomial equalities over commutative semirings/rings was originally presented by Grégoire and Mahboubi [2005]. This is the current implementation of Rocq’s ring tactic, and it was also adopted by Agda’s ring solver [Kidney, 2019]. We’ll explain briefly how the two approaches differ, and then compare the capabilities of both algorithms.

5.1 Reflection

Reflection is using the ability of the Calculus of Constructions to speak and reason about itself. Here is the pipeline for the reflection process used in Rocq described by Grégoire and Mahboubi [2005].

Consider a ring structure A , with two constants 0 and 1, three binary operators $+$, $*$ and $-$ and an opposite unary function $-$. We want to prove the equality of two terms t_1 and t_2 modulo the ring axioms. To do so, they introduce two inductive types PolExpr and Pol , two evaluation functions $\varphi_P : \text{PolExpr} \rightarrow A$ and $\varphi_{PE} : \text{Pol} \rightarrow A$ and a translation function $\mathfrak{T} : A \rightarrow \text{PolExpr}$. PolExpr describes the syntax of terms of type A , while Pol describes normal forms of terms of type A . The reflection process is pictured in Figure 3.

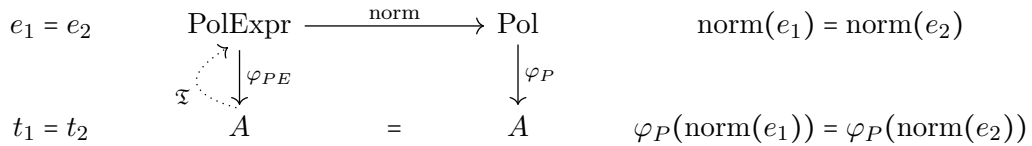


Figure 3: Reflection process

It goes as follows:

1. from t_1 and t_2 , build two corresponding terms e_1 and e_2 with $\mathfrak{T}$.
2. normalise these two polynomials using norm
3. prove the equality $\text{norm}(e_1) = \text{norm}(e_2)$ to conclude for the equality t_1 and t_2 .

In order for this process to be correct, there are some constraints on φ_P , φ_{PE} and $\mathfrak{T}$ that need to be met:

$$\forall e \in \text{PolExpr}, \varphi_{PE}(\text{norm}(e)) = \varphi_P(\text{norm}(e)) \quad (2)$$

$$\forall a \in A, \varphi_{PE}(\mathfrak{T}(a)) = a \quad (3)$$

If the equality of the normal forms hold, it follows that:

$$t_1 = \varphi_{PE}(\mathfrak{T}(t_1)) = \varphi_P(\text{norm}(\mathfrak{T}(t_1))) = \varphi_P(\text{norm}(\mathfrak{T}(t_2))) = \varphi_{PE}(\mathfrak{T}(t_2)) = t_2$$

444 The type `Pol` is built so that equality can be checked syntactically: if two polynomials
 445 $P_1, P_2 \in \text{Pol}$ are equal then so are their syntax tree. Therefore, equality of normal form comes
 446 for free and one only needs to prove the correctness criterion (2) for the tactic to be correct.

447 5.2 Almost-rings

448 The `ring` tactic is capable of dealing both with commutative rings and commutative semirings,
 449 under the same implementation. To unify both structure, they introduce *almost-rings*, an inter-
 450 mediate structure similar to rings and semirings.

451 An *almost-ring* is a commutative semiring completed with an unary operator called *almost-*
 452 *opposite*. The almost-opposite, denoted by $-$, satisfies the following axioms:

$$\begin{aligned} x, y &\vdash -(x * y) = -x * y \\ x, y &\vdash -(x + y) = -x + -y \\ x, y &\vdash x - y = x + -y \end{aligned}$$

453 The last equation defines an associated *pseudo-subtraction*.

454 These axioms do not allow to prove the missing axiom defining the opposite in a ring
 455 $x \vdash x + -x = 0$. This lets them equip a commutative semiring with an almost-ring structure by
 456 taking the almost-opposite to be the identity function, which satisfies the axioms. $\mathbb{N}$ can be seen
 457 as an almost-ring by defining $-x := x$ and $x - y := x + y$. Therefore, implementing a solver for
 458 almost-rings is sufficient to deal with both commutative semirings and commutative rings.

459 Even though the equation $x \vdash x + -x = 0$ is not provable in an almost-ring, it will be proved
 460 for rings by the `ring` tactic as long as $1 + (-1)$ reduces to 0 in the coefficient set used in the types
 461 `Pol` and `PolExpr`.

462 5.3 Soundness and completeness

463 Both the `FREX` solvers and the `ring` tactic are sound and complete, but with a subtlety.

464 Any `fral/frex` is canonically isomorphic to the quotient `fral/frex`, and an effective `fral/frex`
 465 with a constructive universality proof has an effective isomorphism to the quotient setoid. In
 466 this way, a simplifier using an effective `fral/frex` is constructively sound and complete [Allais
 467 et al., 2025]. This completeness is with respect to the underlying theory of the `fral/frex`, not
 468 with respect to all actually provable equalities in the model for which we are trying to prove an
 469 equality. For instance, considering the ring of booleans with logical or as addition and logical
 470 and as multiplication, the addition is idempotent: $\forall x : \text{Bool}, x \vee x = x$. Even though it holds,
 471 the equation is not a consequence of the ring axioms and therefore it is unable to be solved by
 472 a `fral` for rings. Since both addition and multiplication are idempotent for this ring, we are not
 473 under the hypothesis of Theorem 4 and therefore `FREX` cannot be used to solve every equation
 474 over the booleans.

475 The soundness of the `ring` tactic comes from the correctness lemma (2). `ring` computes
 476 the normal forms for the LHS and the RHS of the equation and checks whether they are equal
 477 or not. If they are equal, (2) assures us that the two terms are indeed equal, thus giving us
 478 soundness. Completeness with respect to the commutative semiring/ring theory is ensured by
 479 the validity of the normal form used (sparse Horner forms) and the equation (3). The validity
 480 of the normal forms ensure that if two polynomial expressions represent the same polynomial,
 481 then their normal form will be equal. Equation (3) ensures that the translation process from
 482 expressions to normal forms is correct. However, the equations provable by the `ring` tactic

actually depend on which coefficient set is used in the types `Pol` and `PolExpr`. By default, the `ring` tactic takes $\mathbb{Z}$ as its coefficient set, but it may not always be the best choice. For instance, for solving equations over the ring of booleans, taking the booleans themselves can let `ring` solve equations such as $\forall x : \text{Bool}, x \vee x = x$. This is because $1 + 1 = 1$ for booleans. The left side of the equality is reflected to $X + X$, whose normal form $(1 + 1) \cdot X$ is reduced to $1 \cdot X = X$. Taking the default coefficient set $\mathbb{Z}$ would not allow such equation to be provable for the booleans.

Therefore, both `FREX` and the `ring` tactic are sound and complete with respect to the underlying algebraic theories, but a good choice for the coefficient set of the `ring` tactic can lead to solving more equations.

5.4 Comparison

5.4.1 Supported theories

The `FREX` approach to algebraic simplification lets us tackle many more theories than the `Rocq` `ring` tactic can. In Table 1, we show examples of models for the different distributive combinations of mere/commutative mere/involutive semigroups, monoids and groups. Circled are the theories that the `ring` tactic can handle. Only the commutative semiring/ring corner is covered by reflection-based ring solving, whereas `Frex` extends to semigroup/monoid/group and involutive variants.

The models in the table are the following:

- $\mathbb{N}, \mathbb{Z}, \mathbb{R}$: natural numbers, integers and real numbers with the usual operations.
- $(\mathbb{C}, -)$: complex numbers with conjugation.
- $\mathbb{C}_{>0} := \{a + b\mathbf{i} \mid a \in \mathbb{R}_{>0}, b \in \mathbb{R}\}$ are complex numbers with a strictly positive real part.
- $2\mathbb{Z}[\mathbf{i}] := \{2(a + b\mathbf{i}) \mid a, b \in \mathbb{Z}\}$ with standard addition, multiplication and conjugation as involution.
- $\mathcal{P}(\Sigma^*)$ and variants: formal languages on the alphabet Σ , with union as addition and concatenation as multiplication.
- $M_n(X)$: $n \times n$ square matrices with coefficients in X . $-^T$ denotes matrix transposition.
- $\mathbb{Z}\langle \Sigma^+ \rangle$: finite integer linear combinations of nonempty words, i.e, finitely supported functions from Σ^+ to $\mathbb{Z}$. Addition is pointwise function addition and multiplication is concatenation that distributes over sums. `rev` is word reversal.
- $\text{Rel}(X)$: the set of binary relations on X . Addition is union, and multiplication is composition. Involution is the converse $R^\dagger := \{(y, x) \mid (x, y) \in R\}$.
- $Y_\sigma \llbracket X_1, \dots, X_n \rrbracket$: multivariate generating functions in the variables $X_1, \dots, X_n$ with coefficients in Y . Addition and multiplication are the standard operations, the involution is defined by $f(x_1, \dots, x_n)^{\ast_\sigma} := f(x_{\sigma(1)}, \dots, x_{\sigma(n)})$ where $\sigma \in \mathfrak{S}_n$ is a permutation of order 2, i.e, $\sigma^2 = \text{id}$.
- $\mathbf{m}_n^\sigma := \{f \in \mathbb{N}_\sigma \llbracket X_1, \dots, X_n \rrbracket \mid f(0, \dots, 0) = 0\}$: multivariate generating functions in the variables $X_1, \dots, X_n$ with coefficients in $\mathbb{N}$ and with no constant part.
- $(T_{n, \Delta_{>0}}(\mathbb{R}), -^\dagger)$: upper triangular matrices with coefficients > 0 on the diagonal. Addition and multiplication is standard matrix addition and multiplication. The involution $-^\dagger$ reverses the order of the coefficients on the diagonal, e.g, $\Delta(1, \dots, n)^\dagger = \Delta(n, \dots, 1)$.

Multiplicative structure			Additive structure		
Theory	Comm. ¹	Inv. ³	Comm. Semigroup	Comm. Monoid	Abelian Group
Semigroup	Mere	Mere	$\mathcal{P}(\Sigma^+) \setminus \{\emptyset\}$	$\mathcal{P}(\Sigma^+)$	$\mathbb{Z}\langle\Sigma^+\rangle$
		Inv. ⁴	$(\mathcal{P}(\Sigma^+) \setminus \{\emptyset\}, -^R)$	$(\mathcal{P}(\Sigma^+), -^R)$	$(\mathbb{Z}\langle\Sigma^+\rangle, \text{rev})$
	Comm. ²	Mere	$2\mathbb{N}_{>0}$	$2\mathbb{N}$	$2\mathbb{Z}$
		Inv.	$\mathfrak{m}_n^\sigma \setminus \{0\}$	$\mathfrak{m}_n^\sigma$	$(2\mathbb{Z}[\mathbf{i}], -)$
Monoid	Mere	Mere	$\mathcal{P}(\Sigma^*) \setminus \{\emptyset\}, M_n(\mathbb{N}_{>0})$	$\mathcal{P}(\Sigma^*), M_n(\mathbb{N})$	$M_n(\mathbb{Z})$
		Inv.	$(M_n(\mathbb{N}_{>0}), -^T), (\mathcal{P}(\Sigma^*) \setminus \{\emptyset\}, -^R)$	$(M_n(\mathbb{N}), -^T), (\mathcal{P}(\Sigma^*), -^R), \text{Rel}(X)$	$(M_n(\mathbb{Z}), -^T)$
	Comm.	Mere	$\mathbb{N}_{>0}$	$(\mathbb{N})$	$(\mathbb{Z}, \mathbb{R}, \mathbb{C})$
		Inv.	$\mathbb{N}_\sigma \llbracket X_1, \dots, X_n \rrbracket \setminus \{0\}$	$\mathbb{N}_\sigma \llbracket X_1, \dots, X_n \rrbracket$	$(\mathbb{C}, -), \mathbb{Z}_\sigma \llbracket X_1, \dots, X_n \rrbracket$
Group	Mere	Mere	$T_{n, \Delta_{>0}}(\mathbb{R})$	impossible nontrivially	impossible nontrivially
		Inv.	$(T_{n, \Delta_{>0}}(\mathbb{R}), -^\dagger)$	impossible nontrivially	impossible nontrivially
	Comm.	Mere	$\mathbb{R}_{>0}$	impossible nontrivially	impossible nontrivially
		Inv.	$(\mathbb{C}_{>0}, -)$	impossible nontrivially	impossible nontrivially

¹ Commutativity

² Commutative

³ Involutivity

⁴ Involution

Table 1: Examples for the distributive combination of theories

Some combinations are impossible nontrivially. This is the case of every combination of a multiplicative group with an additive structure that has a 0 element. In such a case, every element is equal to each other. Indeed, suppose that 0 is invertible, i.e, there exists 0^{-1} such that $0 \cdot 0^{-1} = 1$. Because of the distributivity axioms, $x \vdash x \cdot 0 = 0$ holds in all of these cases. We also have the associativity of the multiplication, therefore for every element a :

$$a = a \cdot (0 \cdot 0^{-1}) = (a \cdot 0) \cdot 0^{-1} = 0 \cdot 0^{-1} = 1$$

523 Every element is equal to 1, the only model validating these theories is the trivial model $\{0\}$.

524 5.4.2 Support for additional equalities

525 The Rocq ring tactic supports² taking additional equalities as arguments to solve equalities
526 between terms. This lets one prove goals such as

Goal forall a b : Z, 2 * a * b = 30 -> (a + b) ^ 2 = a ^ 2 + b ^ 2 + 30.
Proof.

²See <http://rocq-prover.org/doc/V9.2.0/refman/addendum/ring.html>.

```

    intros a b H; ring [H].
Qed.

```

527 This is impossible with the FREX approach. The universal property of free algebras and free
528 extensions guarantees that the only equations that are valid in the free algebra/extension are
529 the ones that are provable with the provability relation of Figure 1. Due to the nature of the
530 solving algorithm (§2.4), one can't provide additional facts to the solver, and therefore the goal
531 above is not provable with FREX.

532 However this feature is limited. The supported additional equalities need to be of the form
533 $m = p$ where m is a monomial and p is a polynomial in the corresponding ring structure. There-
534 fore, you can't reason about any other operators than $+$ and $*$, even when giving additional
535 axioms concerning the operator. For instance, define an abstract ring and add an involution
536 operator $\sim$. Even by giving the axioms

```

Rinv_inv : forall x, ~ (~ x) == x.
Rinv_antidistr : forall x y, ~ (x * y) == ~ y * ~ x.

```

537 to the ring tactic, it is unable to solve the basic goal

```

Lemma invInv : forall x, ~ (~ x) == x.
Proof.
  intros.
  Fail ring [(Rinv_inv x) (Rinv_antidistr x)].
Abort.

```

538 because $\sim$ is not a part of the ring structure. This is to be expected when looking at how
539 reflection work in §5.1. Because of this, some theories such as involutive semirings or involutive
540 rings which are supported by FREX can't be supported by the ring tactic.

541 5.4.3 Other FREX capabilities

542 We have presented FREX as a generic algebraic simplification framework, but it is not limited to
543 that role. All of the following capabilities of FREX cannot be achieved by the ring tactic.

544 By appealing to the universal property of the `fral/frex`, every `fral/frex` is isomorphic to the
545 quotient `fral/frex`. This lets us extract a proof certificate, i.e, a derivation for the equation for the
546 provability relation of Figure 1. FREX can package such a derivation as a lemma that is valid for
547 any model of the given theory. Users can then seamlessly invoke these lemmas in any model,
548 avoiding further FREX calls [Allais et al., 2025, Section 5.2].

549 The resulting derivation can also be used to pretty-print human-readable proofs, going through
550 each step and specifying which axiom is used, and where. However, the derivation may not be
551 optimal, leading to possibly large proofs with unnecessary steps. Figure 4 shows the first four
552 solving steps to prove that $(3 + x) + 2 = 5 + x$ in a commutative monoid; these steps aren't part
553 of the optimal derivation involving only four solving steps in total. The full proof is pictured in
554 Figure 5 in the appendix. The obtained derivation can also be used to print Idris proofs for the
555 lemmas [Allais et al., 2025, Appendix C]. Both of these capabilities are not yet possible for proofs
556 involving distributive combination of theories as the printers have not been implemented yet.

557 FREX can also be used to normalise code. Corbyn et al. [2022] present a normalisation
558 by evaluation framework using FREX that can simplify higher-order functional programs with
559 algebraic structure. For instance, the framework can simplify simply typed lambda calculus

$$\begin{array}{c}
\langle \text{Left neutrality} \rangle \quad \quad \quad \langle \text{Left neutrality} \rangle \\
(3 \bullet x) \bullet [2] = (3 \bullet x) \bullet 2 \bullet [\varepsilon] \stackrel{\downarrow}{=} (3 \bullet x) \bullet 2 \bullet \varepsilon \bullet [\varepsilon] = (3 \bullet x) \bullet 2 \bullet \varepsilon \bullet \varepsilon \bullet [\varepsilon] \stackrel{\downarrow}{=} \dots = 5 \bullet x \\
\uparrow \quad \quad \quad \uparrow \\
\langle \text{Right neutrality} \rangle \quad \quad \quad \langle \text{Left neutrality} \rangle
\end{array}$$

Figure 4: Extract of a FREX-extracted proof of $(3 \bullet x) \bullet 2 = 5 \bullet x$ in the additive monoid over natural numbers.

terms extended with an arbitrary commutative monoid M . Take M to be the commutative monoid of integers with multiplication and FREX can normalise terms such as $\lambda x.(2 * 3) * x$ or $\lambda x.(\lambda y.2 * y)(x * 3)$ to $\lambda x.6 * x$, combining beta-reduction and algebraic simplification. While a naive normaliser suffices to reduce the first term to its normal, reducing the second term requires reordering the terms and invoking the axioms of the commutative monoid, which requires FREX.

The FREX simplification can also be applied to multi-stage programming. In [Yallop et al. \[2018, Section 4.3\]](#), FREX is used to simplify the code generated by a staged version of `sprintf` to produce more efficient code requiring less string concatenations, reducing expressions such as `(((" ++ show x) ++ "a") ++ "b") ++ show y` to `show x ++ ("ab" ++ show y)`.

6 Conclusion and prospects

We have found a way to modularly combine free algebras for component theories to get a free algebra for distributive combinations of theories, letting us use the work of [Allais et al. \[2025\]](#) to modularly obtain solvers for varieties of semirings. From monoids and commutative monoids we recover semiring solvers, and from existing involutive constructions we additionally obtain involutive semiring-like solvers. Already 28 non-trivial solvers can be implemented, and performance evaluation indicates that such solvers are suited for use in dependently-typed programming languages.

The use of category theory in this report is semi-implicit, a slightly different result is stated here to keep this report self-contained. The actual construction uses 2-category and especially 2-functors to transport adjunctions, as well as operads and multicategories to capture the distributivity of the multiplicative operations over the additive operations.

It remains to find a construction that works for free extensions. We proved that a modular construction reusing free extensions for the component theories exist. We also managed to narrow down our scope of research to searching for a multicategory suited for free extensions and a particular adjunction.

Other than giving a construction for involutive free extensions, [Allais et al. \[2023\]](#) also presents the theory of multi-frex: free extensions of multi-sorted theories. It allows us to deal with a lot of other theories, such as non-square matrices or linear transformations. However, it has yet to be formalised in Idris. Doing so would let us explore other distributive theories, such as modules, a generalisation of vector spaces in which the field of scalars is replaced by a ring. Modules can be modeled as the distributive combination of said ring over an abelian group.

Another axis of development for the FREX project is the support for second-order/parametrised and essentially algebraic theories. No progress has been made in that direction yet.

Recently, [Fujii et al. \[2026\]](#) gave a canonical construction of a distributive law under some hypothesis in substructural contexts. A future direction is to systematically compare with their work.

References

- Allais, G., Brady, E., Corbyn, N., Kammar, O., & Yallop, J. [2025]. Frex: Dependently typed algebraic simplification. *Proceedings of the ACM on Programming Languages*, 9[ICFP], 30–65. <https://doi.org/10.1145/3747506>
- Allais, G., Corbyn, N., Lindley, S., & Yallop, J. [2023]. Modular construction of multi-sorted free extensions (short paper). <https://api.semanticscholar.org/CorpusID:260005086>
- Beck, J. [1969]. Distributive laws. In B. Eckmann [Ed.], *Seminar on triples and categorical homology theory* [pp. 119–140]. Springer Berlin Heidelberg.
- Bishop, E. [1967]. *Foundations of constructive analysis*. McGraw-Hill.
- Burris, S., & Sankappanavar, H. [1981, January]. *A course in universal algebra* [Vol. 91]. <https://doi.org/10.2307/2322184>
- Corbyn, N. [2021]. *Proof synthesis with free extensions in intensional type theory* [tech. rep.] [MEng Dissertation]. University of Cambridge.
- Corbyn, N., Kammar, O., Lindley, S., Valliappan, N., & Yallop, J. [2022]. Normalization by evaluation with free extensions.
- Fujii, S., Tsai, Y. C., Montacute, Y., & Hasuo, I. [2026]. Monads and Distributive Laws in Substructural Contexts. In C. Faggian & J.-P. Katoen [Eds.], *41st annual symposium on logic in computer science (lics 2026)* [45:1–45:28, Vol. 380]. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. <https://doi.org/10.4230/LIPIcs.LICS.2026.45>
- Keywords: Monad, distributive law, operad, category theory, effect.
- Grégoire, B., & Mahboubi, A. [2005]. Proving equalities in a commutative ring done right in coq. In J. Hurd & T. Melham [Eds.], *Theorem proving in higher order logics* [pp. 98–113]. Springer Berlin Heidelberg.
- Hofmann, M. [1997]. *Extensional constructs in intensional type theory* [C. J. Rijsbergen, Ed.]. Springer-Verlag.
- Hyland, M., & Power, J. [2006]. Discrete lawvere theories and computational effects [Algebra and Coalgebra in Computer Science]. *Theoretical Computer Science*, 366[1], 144–162. <https://doi.org/https://doi.org/10.1016/j.tcs.2006.07.007>
- Kidney, D. O. [2019]. Automatically and efficiently illustrating polynomial equalities in agda. URL: <https://doisinkidney.com/pdfs/bsc-thesis.pdf>.
- MacLane, S. [1971]. *Categories for the working mathematician* [Graduate Texts in Mathematics, Vol. 5]. Springer-Verlag.
- Nielsen, J. [1994]. *Usability engineering*. Morgan Kaufmann Publishers Inc.
- Varacca, D., & Winskel, G. [2006]. Distributing probability over non-determinism. *Mathematical Structures in Computer Science*, 16[1], 87–113. <https://doi.org/10.1017/S0960129505005074>
- Yallop, J., von Glehn, T., & Kammar, O. [2018]. Partially-static data as free extension of algebras. *Proc. ACM Program. Lang.*, 2[ICFP]. <https://doi.org/10.1145/3236795>
- Zwart, M., & Marsden, D. [2018]. Don’t try this at home: No-go theorems for distributive laws. <https://arxiv.org/abs/1811.06460>

635 Appendices

636 A Additional material

Proof (Theorem 1)

3. $\Rightarrow$ 2. and 2. $\Rightarrow$ 1. hold by definition. By cyclic implications, it remains to prove 1. $\Rightarrow$ 3. Let $\langle A, \text{var}_A \rangle$ be a $\mathcal{T}$ -model over X . Suppose that $(FX, \text{var}_{FX}) \models (X \vdash l = r)$. We want to prove that $A \llbracket l \rrbracket \text{var}_A = A \llbracket r \rrbracket \text{var}_A$.

The universal property of FX gives a homomorphism $h : \underline{FX} \rightarrow \underline{A}$ such that $h \circ \text{var}_{FX} = \text{var}_A$.

Lemma 5 (Homomorphisms preserve term semantics)

Let A, B be two $\mathcal{T}$ -algebras, $X \vdash t$ be a term and $h : A \rightarrow B$ be a $\mathcal{T}$ -homomorphism. Then for every function $\text{var} : X \rightarrow \underline{A}$, $h(A \llbracket t \rrbracket \text{var}) = B \llbracket t \rrbracket (h \circ \text{var})$.

Proof

This is a simple induction, since h preserves the semantics of the operators in $\mathcal{T}$. $\square$

Lemma 6 (Homomorphisms preserve equations)

Let A, B be two $\mathcal{T}$ -algebras, $\text{var} : X \rightarrow \underline{A}$ be a function, $X \vdash l = r$ be an equation and $h : A \rightarrow B$ be a $\mathcal{T}$ -homomorphism. Suppose that $A \llbracket l \rrbracket \text{var} = A \llbracket r \rrbracket \text{var}$.

Then $B \llbracket l \rrbracket (h \circ \text{var}) = B \llbracket r \rrbracket (h \circ \text{var})$.

Proof

$$\begin{aligned} B \llbracket l \rrbracket (h \circ \text{var}) &= h(A \llbracket l \rrbracket \text{var}) && \text{since } h \text{ preserves term semantics} \\ &= h(A \llbracket r \rrbracket \text{var}) && \text{since } A \llbracket l \rrbracket \text{var} = A \llbracket r \rrbracket \text{var} \text{ by hypothesis} \\ &= B \llbracket r \rrbracket (h \circ \text{var}) && \text{since } h \text{ preserves term semantics} \end{aligned}$$

$\square$

Now we can finally prove our equality in A . By hypothesis, $FX \llbracket l \rrbracket \text{var}_{FX} = FX \llbracket r \rrbracket \text{var}_{FX}$. Since h is a homomorphism and homomorphisms preserve equations, $A \llbracket l \rrbracket (h \circ \text{var}_{FX}) = A \llbracket r \rrbracket (h \circ \text{var}_{FX})$. But $h \circ \text{var}_{FX} = \text{var}_A$, therefore $A \llbracket l \rrbracket \text{var}_A = A \llbracket r \rrbracket \text{var}_A$. $\square$

Definition 7 (Abelian group monad)

The Abelian group monad $\langle A, \eta^A, \mu^A \rangle$ is given by:

- $A : \mathbf{Set} \rightarrow \mathbf{Set}$ maps a set X to the set containing all finite multisets with elements taken from X and with multiplicities in the integers (i.e, elements of the multiset are allowed to have a negative multiplicity).
- $\eta_X^A : X \rightarrow AX$ maps an element $x \in X$ to the singleton $\{x : 1\}$.
- $\mu_X^A : AAX \rightarrow AX$ takes a union of multisets, accounting for all multiplicities.

$$\mu_X^A \{ \{a : 1, b : -2\} : 1, \{b : 1\} : 3 \} = \{a : (1 \times 1), b : (-2 \times 1 + 1 \times 3)\} = \{a : 1, b : 1\}$$

Definition 8 (Barycentric algebras)

The theory of barycentric algebras **BA** consists of a binary operator $\oplus_p$ for every $p \in [0, 1]$ and the axioms

$x \vdash x \oplus_p x = x$	idempotence
$x, y \vdash x \oplus_p y = y \oplus_{1-p} x$	skew commutativity
$x, y, z \vdash x \oplus_p (y \oplus_r z) = \left(x \oplus_{\frac{p}{p+(1-p)r}} y \right) \oplus_{p+(1-p)r} z$	skew associativity

Definition 9 (Join semi-lattices)

The theory of join semi-lattices **SL** consists of a binary operator $\vee$ and a constant $\perp$, along with the axioms

$x \vdash x \vee \perp = x$	unit
$x, y, z \vdash (x \vee y) \vee z = x \vee (y \vee z)$	associativity
$x, y \vdash x \vee y = y \vee x$	commutativity
$x \vdash x \vee x = x$	idempotence

637 **B Source code**

638 The source code can be found on a fork of the [Idris Frex](https://github.com/S-c-r-a-t-c-h-y/idris-frex) project on GitHub at [https://github.](https://github.com/S-c-r-a-t-c-h-y/idris-frex/tree/amaury)
639 [com/S-c-r-a-t-c-h-y/idris-frex/tree/amaury](https://github.com/S-c-r-a-t-c-h-y/idris-frex/tree/amaury).

640 **C Proof of Theorem 4**

641 The full proof of Theorem 4 exceeds 50 pages and is compiled in a single file on GitHub at
642 <https://github.com/S-c-r-a-t-c-h-y/semiring-solvers/blob/main/proof/semirings.pdf>.

643 **D Research notes**

644 Research notes are available online for completeness on GitHub at [https://github.com/S-c-r-a-t-c-h-y/](https://github.com/S-c-r-a-t-c-h-y/semiring-solvers/blob/main/notes/notes.pdf)
645 [semiring-solvers/blob/main/notes/notes.pdf](https://github.com/S-c-r-a-t-c-h-y/semiring-solvers/blob/main/notes/notes.pdf).

646 **Acknowledgments**

647 I would like to thank Ohad Kammar, my internship supervisor, for guiding me throughout my
648 internship and helping me address any challenges I might have had.

649 I would like to thank Jesse Sigal for helping me with learning category theory, and Justus
650 Matthiesen for helping me deal with some Idris 2 issues.

651 I would also like to thank Sergey Goncharov, Paul B. Levy, Cristina Matache, Sean K. Moss,
652 Dominic P. Mulligan, Gordon D. Plotkin, John Power, Alex K. Simpson and Sam Staton on behalf
653 of Ohad Kammar for fruitful discussions and suggestions prior to my internship.